

THE AI CAN ASK. ONLY THE KERNEL CAN SAY YES.

CertaRig visual explainer script

Target duration 10:30 to 12:00
Original curiosity led engineering narrative

Prepared 13 September 2026

Working title: The AI can ask. Only the kernel can say yes.

Target duration: 10 minutes 30 seconds to 12 minutes

Format: Original curiosity led engineering explainer with visual intuition, clean diagrams, physical demonstrations, and restrained presenter shots. This borrows broad educational techniques, not the exact voice or visual identity of any specific creator.

Production grammar

- Use the presenter on camera for the first 25 seconds, the pivot at 7:45, and the final 30 seconds.
- Use an overhead bench shot whenever physical cause and effect matters.
- Animate one idea at a time. Keep the background quiet and let the safety state use consistent colors: blue for observation, green for permitted, red for forced safe, gold for reset required.
- Never call a simulated trace a hardware trace. Never call a GPIO command transition a measured relay response.
- On first use, define every abbreviation aloud: hardware in the loop, emergency stop, analog to digital converter.

0:00 to 0:35 — The question

Picture: Presenter beside the powered down rig. Tight shot of a potentiometer, then the relay, then the red and green indicators.

Narration:

This small knob can pretend to be pressure. This one can pretend to be flow. And this relay can decide whether a machine is allowed to run.

Now imagine putting an AI in the middle.

The obvious question is whether the AI is smart enough to operate the rig. But that is not the important question. The important question is: who is allowed to say yes?

On screen text: Who is allowed to energize the output?

0:35 to 1:30 — Why a conversational interface is not a safety system

Picture: Animated speech bubble flows toward a numeric threshold. Pause before it reaches the relay.

Narration:

A language model is useful because people do not naturally speak in API calls. We say, “Run the pressure test,” or, “Explain why the rig stopped.” An agent can interpret that request, choose a known procedure, and guide the operator.

But language is flexible. Safety limits must not be flexible. Four point two bar cannot become “roughly five” because a sentence sounded convincing.

So CertaRig separates two jobs. The agent interprets. A deterministic kernel measures, compares, latches, and controls the output.

Visual: Reveal assets/07_product_architecture.png one block at a time.

1:30 to 2:30 — The invariant

Picture: Five switches appear in series. The output lamp is dark.

Narration:

The output turns on only when five conditions are simultaneously true.

Actuation is enabled. A permit has been requested. No trip is latched. The emergency stop loop is closed. And every required process signal is healthy.

Remove any one condition and the output is false.

But the more interesting rule comes after a trip. Returning the pressure to normal does not restart the relay. Releasing the emergency stop does not restart it either. The system must first see healthy inputs, then receive an explicit reset, and only then accept a new permit.

Animation:

```
OUTPUT = enabled ^ permit ^ ¬trip ^ E stop closed ^ healthy
```

Trip one term. Keep the output dark while the term returns. Illuminate only after reset, then permit.

2:30 to 3:20 — What the hardware represents

Picture: Overhead bench image with labels appearing beside components.

Narration:

For the dry bench, two potentiometers emulate field sensors. An ADS1115 analog to digital converter reads them. One channel maps to pressure in bar. The second maps to flow in litres per minute.

The emergency stop has two normally closed paths. One is observed by the software. The other interrupts relay board power independently. GPIO23 commands the relay through a transistor interface. Red means inhibited. Green means permitted.

This is not a hydraulic plant. It is a low voltage hardware in the loop representation of its control and evidence architecture.

On screen label: Mapped dry rig, not hydraulic commissioning

3:20 to 5:30 — The six gates

Picture: assets/01_gate_ladder.png; zoom into each gate.

Narration:

We did not jump from code to an energized relay. We crossed six gates.

Gate zero was read only. We inspected the existing Phase 3 system and confirmed that the new platform had not overwritten it.

Gate one created a sidecar. The product ran from a separate folder, on a separate port, with user local configuration.

Gate two was observe only. Output authority remained disabled while both analog channels were swept. The first attempt failed because the expected pressure movement was not completed and the channel became invalid. We kept that failure. A commissioning system that refuses incomplete evidence is doing its job. The next complete sweep passed with 1,291 samples.

Gate three was the digital twin gate. Six exact procedure hashes first had to pass in simulation.

Gate four allowed controlled actuation. The Fake agent ran five approved procedures, while the kernel owned every comparison and trip.

Gate five restored the original Phase 3 dashboard, verified it was untouched, disabled actuation, and halted the Pi.

Pause on screen: Observe → simulate → arm → run → restore

5:30 to 7:45 — What actually happened in the tests

Picture: Hardware shot, dashboard capture, then Figures 2, 3, and 5.

Narration:

After the initial failed observation, six hardware runs passed. Together they recorded 3,422 samples and passed 34 evaluated checks.

The relay truth table repeated reset, permit, and safe, then deliberately tried to request a permit without reset. The kernel rejected it.

In the pressure procedure, the signal crossed the 4.2 bar guardrail and peaked at 4.83 bar. The commanded output went safe and stayed latched.

The flow procedure crossed 15 litres per minute and peaked at 15.41. Again, the output went safe and a new permit was rejected until reset.

The dual input procedure did both independently. Pressure tripped first. We lowered it, reset, and requested a new permit. Then flow tripped the same output.

Finally, the emergency stop opened the safety loop. The kernel recorded forced safe. Releasing the button produced reset required, not restart. A permit request before reset was rejected.

The recorded software transitions were all under one millisecond. That is useful evidence about the kernel's state change, but it is not a measurement of mechanical relay latency because this bench has no contact feedback sensor.

Visual sequence:

1. assets/02_hardware_peaks.png
2. assets/05_kernel_event_timeline.png
3. assets/03_software_response_time.png

7:45 to 9:05 — The pivot

Picture: Presenter returns. The bench dissolves into a generic rig map.

Narration:

This changed how I think about the product.

The tempting pitch is that an agent can look at any unknown machine and operate it. We have not proved that, and more importantly, that is not where the strongest value is yet.

The better product is an evidence first commissioning runner for mapped rigs.

A user describes the modules, signals, units, limits, and anything that must remain observe only. The platform proposes a configuration. A human reviews and applies it. The twin rehearses the exact procedure. Only then can an explicitly armed kernel reach an output.

The agent makes the workflow understandable. It does not become the safety authority.

On screen product statement:

Agent guided commissioning. Deterministic control. Inspectable evidence.

9:05 to 10:15 — Why the evidence bundle matters

Picture: Animate a run folder assembling: procedure, events, run result, report, manifest, checksums.

Narration:

A demonstration can look convincing and still be difficult to audit. CertaRig treats evidence as an output of the procedure itself.

Each run records the rig identity, procedure hash, contract hash, steps, kernel events, checks, peaks, sample count, result, and file checksums. The seven exported hardware archives were rehashed during this analysis, and every file listed in their export manifests matched.

There is also a gap. The exports reference each raw hardware CSV by filename, row count, and hash, but the CSV itself is missing from the pulled bundle. The next correction is straightforward: include the recording in the archive and add a test that verifies its hash before an export can pass.

That is the point of evidence first engineering. A limitation is not hidden. It becomes the next testable requirement.

10:15 to 11:10 — What comes next

Picture: Three steps appear: second simulated plant, commissioning interview, feedback sensor.

Narration:

The next milestone has three parts.

First, build a thin commissioning interview that proposes a rig configuration but cannot apply it without a person.

Second, prove that workflow on a second simulated plant, such as a thermal process, before touching another physical output.

Third, add isolated relay feedback or current sensing so we can measure the full electrical response, not only the software command.

Only after those tests should we expand the claim from one mapped bench to a repeatable commissioning platform.

11:10 to 11:40 — Conclusion

Picture: Presenter, then the green indicator briefly turns on under a controlled permit and returns to red safe. Use existing recorded footage if available; do not recreate an energized sequence solely for the edit.

Narration:

The most important result is not that an AI switched a relay.

It is that the AI never had to own the dangerous decision.

CertaRig let the agent ask, the operator approve, the kernel decide, and the evidence explain what happened afterward.

That is how a conversational system begins to earn a place beside real engineering hardware.

Final card: The AI can ask. Only the kernel can say yes.

Recording and edit checklist

Capture or reuse the following footage:

1. Presenter hook and conclusion in the same camera setup.
2. Clean overhead shot of the complete unpowered bench.
3. Close shots of both potentiometers, ADS1115, emergency stop, relay, and red and green indicators.
4. Studio Live page showing both channels and trip lines.
5. Procedure page starting relay truth table, pressure, flow, dual input, and emergency stop tests.
6. Evidence page opening one completed run and its checksum information.
7. Repository tree showing certarig/edge, studio, skills, config, and evidence.
8. Screen recording of the simulator pressure and flow procedures.

Use the seven generated figures in docs/explainer/assets/ as full screen inserts. Keep each figure visible long enough to read its title and one central takeaway.